UI BUILDER: An Automatic Code Generation tool for design & implementation of User Interface

Keshawa Ekanayake
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it19150758@my.sliit.lk

Sakthivel Sanjay
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it19158228@my.sliit.lk

Kasuni Navodya
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it19144986@my.sliit.lk

Kavindi Gimshani
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it19150826@my.sliit.lk

Dr. Nuwan Kodagoda
Associate Dean
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
nuwan.k@sliit.lk

Ms. Kushnara Suriyawansa
Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
kushnara.s@sliit.lk

***Abstract*— With the advancement of technology, the growth of software applications has rapidly increased. Both small-scaled businesses and large-scale corporations prefer software applications for achieving their business objectives such as marketing, automation, management, etc. With changing business requirements, the software application had to scale in the direction of adding new features or improving its non-functional requirements. In many aspects, scaling results in increasing the code size and complexity of the application. The question arises on how to improve the software development process to withstand changing business requirements. Automatic Code Generation in Automatic Programming is a suitable solution. This paper introduces a new tool "UI Builder" which can convert user interface (UI) sketches to code. The use of a code generator increases the productivity and efficiency of the organization. The tool satisfies different needs of developers using its enhanced features and easily understandable user flow. With critical experimentation, the effectiveness of the tool is compared with manual implementation to highlight the benefits and advantages. A bug-free, structured, and properly organized code was generated using the tool. The Model used in the tool achieves 91.0% accuracy and 93.0% validation accuracy. The generated code showed 92.0% improvement in average development time and 87.5% improvement in code size.**

Keywords—Software application, Automatic Code Generation (ACG), Automatic Programming, Code generator.

I. INTRODUCTION

Automatic Code Generation (ACG) plays an important role in the software engineering process which introduces new ways and trends in the software development process [1]. ACG marks to be the most trending topic among software developers who involve in continuous software development. Making the Software Development Life cycle (SDLC) process flexible, platform-independent, and auto-generated code with globally accepted standards is proposed by the Object Management Group (OMG) [2]. The use of websites in the direction of marketing products and services has increased the need for rapid development with improved efficiency.

A website involves developing frontend and backend code with database connectivity which results in delivering the completed application to the end-user through different iterative cycles. The software development process involves requirement gathering, designing, developing, testing, and delivering software products but with growing user expectations and advancements in technology, end-users and customers expect the software development process to be flexible and cater to their requirements and expectation at any stage of development. Considering the efficiency of software developers who are involved in the software development process where they need to rewrite the same code in different modules in the application which requires more time and reduces the readability of code. Refactoring the written code, commenting, and checking for bugs becomes a nightmare for developers which in turn results in increased costs for the organization. This reveals an open-ended problem that can be explored to provide a better solution.

Our study focuses on Automatic Code Generation (ACG) using code generators that touch base on the concepts of automatic programming [3][4]. ACG is the process of generating both frontend and backend code with database connectivity automatically for software applications using different forms of inputs from user. The output can result in source code files, configuration files, etc. with database records and validations.

Using ACG in application development gives several advantages for software developers and organization. An increase in productivity, code quality, and efficiency are a few advantages which can be highlighted. The benefits of using the ACG process in an organization can be listed as follows:

1. Prevent code rewriting and promote code reuse in different modules of application.
2. Break the categorization among software developers as backend and frontend developers, while bringing them into one common path.
3. Hire a few developers and educate them about the code generator tool. This reduces the cost for the organization.
4. Developers' time is saved which they need to verify and validate the generated code for the given business logic.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

5. Automating the code implementation becomes a milestone in software development.

In this paper, we introduce a new tool named "UI Builder" which can auto-generate code for the given User Interface (UI) sketch. The tool has 4 major components which are (1). Converting UI sketch to standard UI, (2). Generating web frontend code, (3). Generating mobile frontend code (4). Generating backend code. Using Machine Learning (ML) model different UI components are identified. We created our own dataset with different UI components and the ways they could be drawn. A survey conducted among developers and Software Engineering undergraduates helped to get required data. eXtensible Markup Language (XML) was used to document the UI specifications. The code generation process used this XML file to generate code. Backend code generation used a refined form of this XML file to document the table structure and relationships.

Finally, a UI image with structured backend and frontend code is obtained. The highlights of this paper can be listed as follows:

1. A new tool named UI Builder is introduced.
2. The ACG process using the tool is explained.
3. The learning outcomes and areas for development are identified and discussed.
4. Explains the evaluation process of the generated code along with different experiments conducted.

The rest of the paper is organized as follows: Section II explains the similar studies with literature. Section III describes the methodology and how ACG is achieved using the tool. Section IV provides the details of the evaluation and results. Finally, the paper concludes with Section V which highlights the conclusion from the study and future work.

II. Related Work

In 2020 Sermuga Pandian and Vinoth Pandian in their research paper [5], expressed the ideas about how to convert low fidelity UI sketches to higher fidelity UI sketches. This research paper mentioned Artificial Intelligence (AI) can be used to identify UI sketches. An AI tool named "MetaMorph" was introduced in this research which is used to detect constituent UI elements of low-fidelity sketches. The use of synthetic low-fidelity UI sketches and DNN UI element detector can be highlighted in this research.

The research paper published by Chowdhury, Anirban in 2019 [6], explains the problems related to the rough sketches drawn by pen or pencil. The aim of this research paper is to design and develop UI stencils to help mobile UI designers to create quick, easy, and visually pleasing paper prototypes. For this research, a total of three UI stencil prototypes were designed and developed for usability testing to ensure the user acceptance of UI stencils. The researcher observed that users were taken comparatively less time for UI screens using 'Stencil-3' than the time taken for hand drawings of the same UI screens.

In 2021 the research paper [7] explained the process of converting visual sketches into code. The tool was named 'Visual Sketching' which uses computer vision algorithms and ML techniques to convert visual sketches to code. The approach used in this research mostly fits data scientists, but it is said that the tool can be practiced in any domain. The researchers found the lack of research on the use of images to compile code snippets as an immediate gap that can be filled. They focused on this idea throughout the research paper by introducing novel approaches and displaying a higher level of accuracy in generated code. The researchers briefly described the capabilities of the tool under different conditions along with future research work.

In the year 2020, a template-based code generator for web application was published in the Turkish journal of electrical engineering and computer sciences [8]. This paper identifies the problems faced by businesses from small scale to large scale in their need to improve the software development process of web applications. The introduced code generator was integrated with a real-life web application. Experiments were conducted to prove the effective integration of the introduced code generator with real-life web applications. The authors suggested the use of code generators which increases the developer productivity and efficiency.

This research paper, published in 2016, introduces a new ACG tool called Rapid Project Builder (RPB) [9]. This RPB is designed using Java and it is an open-source tool. It stores the captured UI elements as XML metafiles and generates automated code. Features found here include automatic generation of code using specialized languages such as C ++, Java, and C #, allowing users to create a UI. The tool used Drag and Drop (DND) approach with the source code file containing the complete functioning code with BL.

This research paper, published by JonnadulaNarasimha Rao, GajulaHarish in 2020, [10] introduces a pre-programmed web page implementation for mockup images. It uses hand-drawn mock-ups as input and gives structured HTML code as an outcome. Here the REMAUI algorithm is used to recognize GUI components such as buttons and text boxes. MYSQL database is used to store uploaded images as input. The concepts used in this tool are Object detection and pruning (cropping), Object recognition, Sketch, and HTML Builder. According to this research paper, the resulting application had 90% accuracy and as an extra feature, they also implemented the colors of the buttons correctly.

The authors in [11] have done research based on the MDA approach to generate code for mobile applications. The suggested approach for the code generation was explained ways to generate Java code from UML diagrams (class, sequence diagrams) and model the code. The proposed solution resulted in generation of behavioral code which is based on sequence diagrams and for that reason it generates lines describing method invocations. As the output of this research, it automatically generates Java-Android and C# codes, according to the specified application model and target platform. This research mentions that the development of Android applications or Windows applications has its own features, which should be considered during modeling and code generation.

III. METHODLOGY

A. *UI Generator*

This component mainly focuses on identifying different UI components and converting to standard UI with XML file to document UI specifications. This research component consists of both functional and non-functional requirements. The output of this research component is to obtain a standard UI for given rough sketch. The users expect to have a standard UI for the rough sketches that they have uploaded**.** The main features of this research component can be listed as follows:

- Developers can get a standard UI for the given rough sketch.
- Ability to design the UIs within a short period of time.
- User-friendly interfaces.
- Reliable, creative, and faster standard UI generating process

The developer gets the opportunity to design the standard UIs for both mobile and web. To work on the desired process, the user needs to select which UI model that the user needs. As an example, if the user needs a UI for web design, he/she needs to select the "WEB" option in in the application. For a mobile design, the same strategy is used.

The process of the generating standard UI for the given rough sketch can be listed as follows,

- The user needs to upload a rough sketch to the application. – Only work done by the user.
- Identifying the UI elements in the rough sketches using a Convolutional Neural Network (CNN) model.
- Process the UI elements using an image processing technique and develops an XML document.
- Map the rough UI elements to standard UIs.
- Design the standard UI.

1. Upload rough sketch to the application.

Initially the user needs to upload the rough sketches to the application. Users need to draw rough sketches by hand, or any other tool, and those rough sketches need to be uploaded to the application. Figure III.1 shows sample rough sketches that was uploaded to the tool.

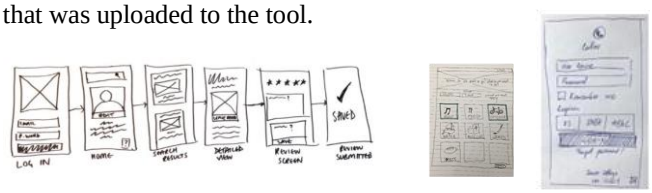


Figure III.1: Sample Rough Sketches

2. Identify UI elements in rough sketches using CNN model.

To identify UI elements in the rough sketches we have created a model using python TensorFlow and we have collected a dataset from a survey. After collecting the dataset, the model was trained and tested using the dataset.

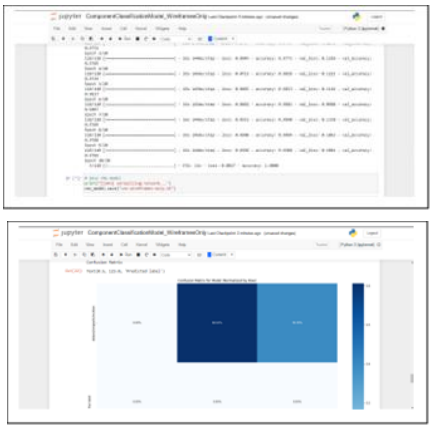


Figure. III.2. Train the model & Test the database

3. Use image processing technique to segregate UI elements and develop an XML document.

To extract the UI elements in the rough sketches we need to use a better image processing strategy. The image segmentation in OpenCV is used to extract the UI elements in rough sketches. For UI elements identification, we used OpenCV and for the label identification of rough sketches, we used Google Vision API. By using these two strategies we managed to identify the UI elements in the rough sketches.

- Image extraction: Image segmentation in OpenCV.
- Label extraction: Google Vision API.

4. Map the rough UI elements to standard UIs.

Mapping the UI rough sketches to the standard UI elements is achieved in this phase. From the resulted XML document, UI elements from rough sketch was mapped to standard UIs. A simple demonstration of this process is given below:

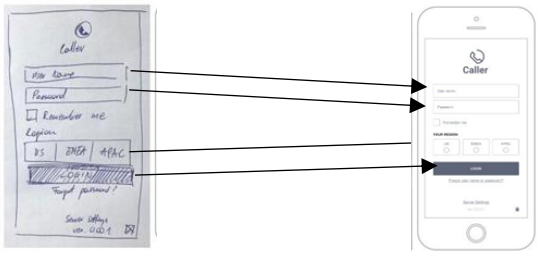


Figure. III.3. Mapping UI sketches to standard UI

5. Design the standard UI.

This is the final phase of this research component. In this phase, the users can have the standard UI for given UI rough sketch. This research component results in an XML document and a standard UI for a given rough sketch.

B. *Frontend Code Generation - Web*

Recently, with rapid enhancement of technology, automated code-generating tools are invented and widely used in the industry. The front end of the tool will be more important to get interest from developers. Competitively,

there are many frontend code generation tools that do automated code generation, but the following describes the novelty and uniqueness of web frontend code implementation in our tool over existing frontend tools.

1. Key features of the Web Frontend code generator.

- Utilization of high performance and high demand MERN Stack technologies for implementation.
- Pre-defined template styles that can be customized based on user requirements.
- Display generated frontend code with selected UI template.
- Accuracy and user-friendliness.

2. Key technologies.

The main technology used for frontend code generator is MERN stack. ReactJS for frontend implementation, NodeJS and ExpressJS for backend implementation, and MongoDB as the database. Automated code-generating tools in the market mainly focused on generating HTML code. HTML is simpler but less secure language with limited security features than ReactJS. The ReactJS library is more popular than HTML in web frontend development.

3. Component Overall Design

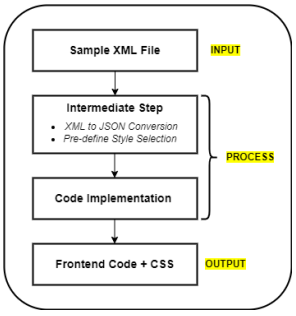


Figure. III.4. Component Overall Design

The above figure presents the overall architecture of the used to implement web frontend code generator. The input is a sample XML file, and the output is generated web frontend code with embedded style sheet. The code generation process consists of two parts as follows:

- **Intermediate step**: The main function of this step is to convert the input XML code to JSON code and create pre-defined customizable template styles.
- **Code implementation**: This is the main process responsible for generating web frontend code.

4. Component Implementation.

4.1. Sample XML File

The XML File acts as the input for web frontend code generator component. It contains the template widget details, their types, names, and alignments. These types are important for identifying the styles associated with each widget detail. Examples of those details are shown in the table III.1 below.

Widget	Type	Name
Form	form	Form
Label	header	FormHeading
	label	usernameText
Separator	horizontal line	FormSeperator
TextBox	input	usrTextBox
Button	button	submitBtn

Table. III.1. Identify the XML widgets and their details

4.2. Convert XML to JSON

The sample XML file obtained as input is converted into a formatted into JSON. This makes it more readable since details on JSON file are easier to read than details on XML file which is used in React implementation.

4.3. Pre-define template styles selection

A special feature that can be seen in this component is the ability to select template styles. Here the system provides five pre-defined template styles that allow the users to select according to their own preference. The tool also allows the user to add or modify styles. The user can view the predefined template in full screen simply by hovering the cursor over the desired template.



Figure. III.5. Pre-define template stypes

4.4. Generate & display web frontend code

This describes the web frontend implementation process focusing on the approach used for code generation. React frontend development is done using converted JSON details and stored style details. The basic considerations in this implementation are as follows.

- The code should be simple and accurate.
- Use meaningful and comprehensive comments.
- Use proper coding standards and best practices.

At the end of the frontend code generation, the user will be able to retrieve the generated code and the resulting UI template. The user can view the generated code in the desired mode either white/light mode or dark mode.

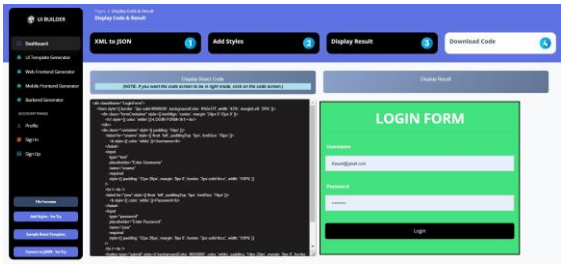


Figure. III.6. Display Code & Result UI

4.5. Download the generated frontend code

Finally, the generated web frontend code is downloaded to the local machine. The generated React frontend code which contains stylesheet can also be downloaded as a JavaScript (.js) file.

C. Frontend Code Generation – Mobile

The mobile front-end code generator fulfills user requirements of mobile code generation by few clicks to fill the gap in mobile front-end code generation of the existing solutions. It results in Flutter code for UI sketch which the user has provided as input to the system. Due to the capabilities of this component users can develop the mobile code for a specific UI. This system is suitable for even people with limited programming or development experience and knowledge that they have in mobile application development.

The output code will be generated using Flutter as language. The reason to choose Flutter as the language is, open-source software development kit (SDK) for cross-platform development capability. It fills the main gap of the current solutions for the mobile application development which is not suitable for the cross-platform mobile application. The procedure for the mobile application frontend code generation could be listed as follows.

1. Read & extract UI elements from XML File.

The mentioned XML file will be the input for this component. As the UI elements are mentioned in the XML file, this component reads the UI elements name and other reverent details which are required for the mobile frontend code generation.

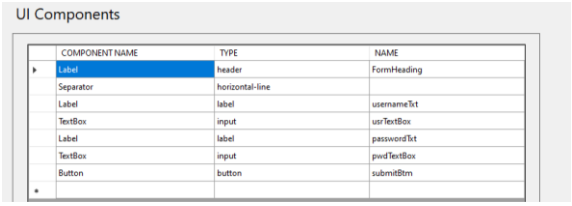


Figure. III.7. Read and extract UI elements

2. Color Theme

The user is given the opportunity to choose the color theme for the interface which they intend to generate code. The system gives the options of pre-defined color themes for the user and the user can select one theme for the interface. Each theme and the respective colors for the UI elements are stored in the system. When the user selects a theme, the stored details will be called for the code generation process.

3. Code Generation

As the final step of mobile frontend code generation, Flutter code for the interface will be developed according to the color theme that the user has selected. The code for each element is stored in the system. During this step, UI elements will be mapped with the code segments. After going through all the elements of the interface, the whole code for the interface will be completed.

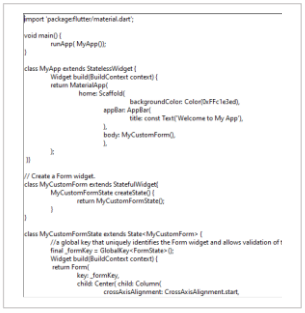


Figure. III.8. Generated mobile code

D. Backend Code generator.

This component gets more prioritized in this tool due to its functional and non-functional capabilities. Developers expect a complete application with frontend and backend code together, but the existing tools don't provide this feature in its implementation. Our tool becomes a solution, and this component focuses on auto-generating backend code for the given UI sketch. The major features of this component can be given as follows:

- Developers get the opportunity to choose between databases. (Microsoft SQL Server, MongoDB)
- Code generation process through button clicks.
- Generated code has a similar texture to real handwritten code.
- Reliable, accurate, and relatively faster code generation process compared to manual implementation.

The developer gets the opportunity to generate Create, Retrieve, Update and Delete (CRUD) code with MS SQL Server database and REST API Code generation with MongoDB. The component can function independently where the developer can generate backend code alone in the absence of frontend code. The processes involved in the code generation process can be given as follows:

- Developers choose between the two databases.
- The code generation process is initiated by developers. Developers create a database in MS SQL Server or cluster in MongoDB.

1. Converting XML to refined XML File.

A refined XML is used in this component which varies from the original XML file obtained from the UI generator component in a way, it describes the table structure and relationships. The refined XML is constructed by reading the original XML file and mapping them into relevant table columns. Figure III.9 shows how a sample refined XML file generated from the tool.



Figure. III.9. Refined XML File

2. Database Connection and Table Creation.

The developer started the code generation process by creating a database in MS SQL Server or creating a cluster on MongoDB. The refined XML (explained in section 1) is mainly used in the table creation process. Any modifications made by the developer on the table structure is validated and saved on the refined XML file, which can be used in the future.

2.1. Refined XML for Table creation on MS SQL Server.

The refined XML file is read and processed to identify the table name, column names, data types, and key fields. During the table creation process, the above details are loaded into a data table and the developer is allowed to modify if necessary. The details are validated and upon successful table creation, the SQL query used for the process is displayed for developers.

2.2 Refined XML for Document creation on MongoDB.

A similar approach described in section 2.1 is used for document creation on MongoDB. The refined XML is read and mapped to a pre-defined model class template. Which results in code implementation of model class with a schema name, columns, and data types.

3. CRUD Code Generator.

Following the table creation in MS SQL Server, the backend code generator focuses on generating CRUD code. The CRUD operation names are auto populated depending on the table name. For example, if the table name is "Customer" then the CRUD operation name will be auto-populated as "AddCustomer", "GetCustomers", "UpdateCustomerById" and "deleteCustomerById". The developer can modify the auto-populated CRUD operation name and the component validates these modifications.

Once the developer checks on the relevant CRUD operation checkboxes, the code is generated in real-time. The developer can view the generated code from the application and download the code to their own PC.

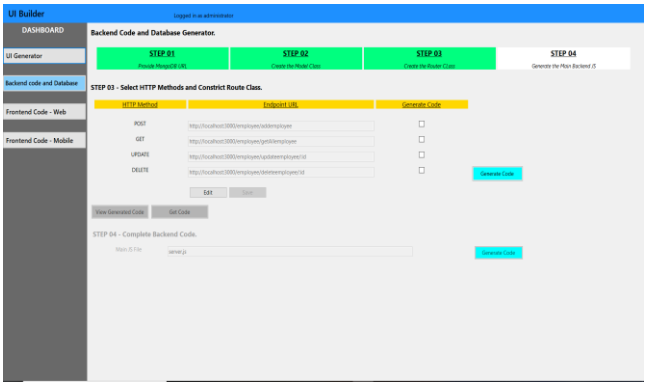


Figure. III.10. UI for CRUD Code generation in UI Builder tool

4. REST API Code Generation.

Once the MongoDB URL and port number is saved successfully, the model class needs to be generated. The refined XML is read and processed to generate the model class. Upon successful generation of the model class, code related to the REST API endpoint URL will be generated. For example, consider "CustomerModel" as the model class name, for which the endpoint URL will be autogenerated as given in the following table III.2.

HTTP	Endpoint URL
POST	http://localhost:3000/customer/addCustomer
GET	http://localhost:3000/customer/getAllCustomers
PUT	http://localhost:3000/customer/updateCustomerById/:id
DELETE	http://localhost:3000/customer/deleteCustomerById/: id

Table. III.2. List of auto-generated endpoint URL

The MongoDB URL and port are bind to relevant variables and the generated REST API route class is called to make the endpoints functional. The REST API Code generation completes with generating the main backend JS file and the developer obtains all the generated code as visualized in the following figure III.11.

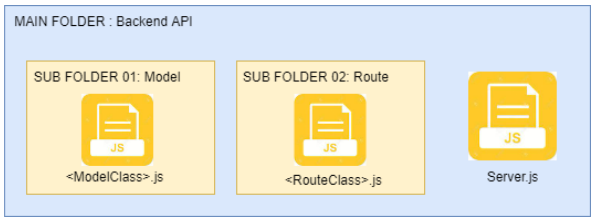


Figure. III.11. Output folder structure for REST API Code

IV. RESULTS & DISCUSSION

After the development of tool with its capability of automatically generating frontend and backend code, we focused on evaluating and finding the accuracy of the generated UI and code for the given UI sketch given as an input. Unlike a machine learning model where finding the accuracy is an easy task, determining the accuracy of generated code was challenging. Through several experiments and methods used by researchers, we decided to use a custom evaluation metric where it is efficient in a way the manual and auto-generated code can be compared.

The experiment was conducted among developers selected from a leading software company and undergraduates in our university. We selected a senior developer and a junior developer, and they were asked to develop selected modules (Explained in an upcoming paragraph) both manually and using our UI Builder tool. The experiment had 3 major phases which are:

1. The developers implemented the given modules both manually and using our tool.
2. Data obtained from phase 01 were analyzed and mapped to the evaluation metric.
3. The evaluation metric was critically analyzed.

First, we selected some important modules which highlight and cover the functionalities of our tool. (1). Employee Registration Form, (2). Customer Signup in E-commerce site, (3). IT Asset Request, (4). Vehicle Registration for reputed Automobile Company are some of the samples among ten selected modules which were used as test subjects. The developers used our tool to generate both frontend code and backend code in their first attempt. Then they developed the modules manually using the same technology on which they generated the code.

Secondly, we formulated an evaluation metric to describe the accuracy and reliability of the generated code. The following table IV.1 describes the factors considered in the evaluation metric which are common principles considered during a code review.

DEVELOPMENT TIME	This describes the time obtained for module development.
BUG COUNT	The number of bugs present on code.
TEST RUN	The test results for the code written or generated.
CODE SIZE	The number of lines the code is written.
CODE QUALITY	The coding standards and best practices are injected in the code.

Table. IV.1. List of auto-generated endpoint URL

A similar metric was used by Akbulut and Toprak [12] and Possatto and Lucrédio [13] which they used for their study. Test run and the Development time mainly impact the efficiency of the code generation tool. Bug count provides evidence on the reliability and un-detected functional breaks which were left un-recognized during test run. The Lines of Code (LOC) and code quality helps to understand the code improvement and best practices injected in the code.

Finally, the data obtained from the first phase was mapped into the evaluation metric mentioned in phase 02. Thereafter the results obtained from the code implemented both manually and using the tool were compared and processed to determine the accuracy of the generated code.

A. Results

The evaluation metrics and the data gathered for each UI template are displayed in figure IV.1. The column Man. represents the date related to manual development and the column Auto. represents the data related to automated development.

	Development time (minutes)		Bug Count		Test Run		Code Size (LOC)	
	Man.	Auto.	Man.	Auto.	Man.	Auto.	Man.	Auto.
Employee Registration Form	350	5	20	0	15	3	320	210
Customer Signup in E-commerce site	480	4	12	0	25	2	423	324
IT Asset Request	620	8	16	2	17	2	210	84
Vehicle Registration for reputed Automobile Company	500	7	11	0	27	2	353	294
Login Form	400	4	3	0	20	3	242	159
Homepage of E-commerce site (List of products displayed).	650	8	14	0	32	2	510	415

Figure. IV.1. UI for CRUD Code generation in UI Builder tool

The first metric is development time. The developers had to go through the UI sketch and develop the code manually. Therefore, the time they took for the development is more than the time that UI Builder took for the development. As the manual code implementation had to be done each line by line, within a few buttons clicks the code can be generated. The results of figure IV.1. clearly shows the benefits of the automated tool as it saves a large portion of development time.

Bug count is the second metric that is used for the evaluation. As the same format is used and the validations are guaranteed in the automated generation it will not be missed during the development. Therefore, the number of bugs is lesser than the manual development. When a bug is found the system could be updated. Therefore, there is a guarantee that that same bug issue will not be raised again in the system.

The third metric is Test Run. In manual development, the developer must do the test run each time when the code is modified. The whole code is generated at once in UI Builder. Therefore, one test run was enough to achieve the results in figure IV.1. The time for the test run will not be wasted and it will be limited to one time in the automated system as the code is developed at once.

The fourth metric, the number of lines in the code depends on the developer in the manual system. In UI Builder the number of lines depends on the number of components in UI. The code is stored in the system, and it maps to the UI. Therefore, the number of lines for each component is fixed in the automated system. The number of code lines are lesser in the automated development than the manual development.

Analyzing the results of the evaluation that is shown in figure IV.1. we can reach to a conclusion of the reasons why automated development is useful and productive than the manual development.

V. CONCLUSION

The functionalities used for An Automatic Code Generation tool for the design and implementation of User Interfaces were discussed in this paper. It has been proven that achieving the goal of the system was succeeded through these functionalities.

In this study, a UI Builder tool was developed that takes hand-drawn rough UI sketches and gives a well-structured web frontend code, mobile frontend code, and backend code. To that end, a dataset consisting of images containing various hand-drawn UI sketches of web and mobile page designs was used. Currently, the system generates output using a bounded number of technologies. As planned for future work, we hope to generate frontend and backend codes according to other technologies that users prefer.

REFERENCES

[1] H. Liao, J. Jiang and Y. Zhang, "A Study of Automatic Code Generation," 2010 International Conference on Computational and Information Sciences, 2010, pp. 689-691, doi: 10.1109/ICCIS.2010.171.

[2] Omar EL BEGGAR, Brahim BOUSETTA and Taoufiq GADI, "Automatic code generation by model transformation from sequence diagram of system's internal behavior", International Journal of Computer and Information Technology, vol. 01, no. 02, November 2012

[3] Rich C, Waters RC. Automatic programming: myths and prospects. Computer 1988; 21 (8): 40–51. doi: 10.1109/2.75

[4] Syriani E, Luhunu L, Sahraoui H. Systematic mapping study of template-based code generation. Computer Lan-

[5] Sermuga Pandian, Vinoth Pandian, et al. "MetaMorph: AI Assistance to Transform Lo-Fi Sketches to Higher Fidelities." 32nd Australian Conference on Human-Computer Interaction. 2020.

[6] Chowdhury, Anirban. "Design and development of a stencil for mobile user interface (UI) design." Research into Design for a Connected World. Springer, Singapore, 2019. 629-639.

[7] M. d'Amorim, R. Abreu and C. Mello, "Visual Sketching: From Image Sketches to Code," 2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2020, pp. 101-104.

[8] Elec, Turk & Sci, Comp & Uyanik, Burak & Şahin, Veysel. (2020). A template-based code generator for web applications. Turkish Journal of Electrical Engineering and Computer Sciences. 28. 1747 – 1762. 10.3906/elk-1910-44.

[9] S. D. Rathod, "Automatic code generation with business logic by capturing attributes from user interface via XML," 2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT), 2016, pp. 1480-1484, doi: 10.1109/ICEEOT.2016.7754929.

[10] JonnadulaNarasimha Rao, GajulaHarish, "PRE-PROGRAMMED WEB PAGE IMPLEMENTATION FOR MOCKUP IMAGE" http://www.joics.org/gallery/ics[1]2878%20-%201.pdf [Accessed: 20 – Jan - 2022]

[11] Parada, Abilio, Milena Marques, and Lisane B. de Brisolara. "Automating mobile application development: UML-based code generation for Android and Windows Phone." Revista de Informática Teórica e Aplicada 22.2 (2015): 31-50. Phone, Abilio G. Parada, Milena R. S. Marques, Lisane B. de Brisolara

[12] Akbulut A, Toprak S. Code generator framework for smart TV platforms. IET Software 2019; 13 (4): 268–279. doi: 10.1049/iet-sen.2018.5157

[13] Possatto MA, Lucrédio D. Automatically propagating changes from reference implementations to code generation templates. Information and Software Technology 2015; 67: 65–78. doi: 10.1016/j.infsof.2015.06.009